Chapter 3 — Semantic HTML and Accessibility

A page can look perfect and still be unusable for someone who navigates by keyboard, listens through a **screen reader** (software that reads the page aloud and lets blind users move around it with keys), zooms to 400 %, or can't tell red from green. **Accessibility** (*a11y*: "a", 11 letters, "y") means building pages that work for all of them, and the most effective tool is **semantic HTML**: using the element whose meaning matches the content. This chapter covers the *accessibility tree*, landmarks, headings, alt text, ARIA, focus management, form labels, WCAG 2.2 and testing.

In this chapter: 3.1 Why semantics and accessibility · 3.2 Landmarks · 3.3 Headings and outline · 3.4 The accessibility tree · 3.5 Alt text · 3.6 ARIA · 3.7 Keyboard and focus · 3.8 Accessible forms · 3.9 WCAG 2.2 · 3.10 Testing tools.

Code is in `code/`. Most scripts open the `.html` files in **real headless Chrome** via `puppeteer-core` (a Node.js library that remote-controls Chrome; helper `code/a11y_lib.mjs`) and print Chrome's own accessibility tree — the data a screen reader receives. `06_mini_audit.mjs` uses jsdom and `dom-accessibility-api`.

3.1 Why semantics and accessibility

What it is

Semantic HTML = choosing elements for what content *is* (`<button>` for a button, `<nav>` for navigation, `<h2>` for a section heading), not how it looks. **Accessibility** = people with disabilities can perceive, understand, navigate and interact with the page. They are linked because browsers expose your HTML's semantics to **assistive technologies** (AT): screen readers, screen magnifiers, voice control, switch devices (one or two big buttons replacing keyboard and mouse), braille displays (raised pins showing text as braille). A `<div>` styled as a button is a painted-on door.

Who benefits

Group	Examples	What they rely on
Blind and low-vision users	Screen readers (JAWS, NVDA, VoiceOver, TalkBack), magnifiers, high zoom	Roles and names, headings, alt text, contrast (text vs background brightness), reflow (content rewrapping at high zoom)
Motor impairments	Keyboard only, switch access, voice control ("click Submit")	Focusable controls, visible focus, big targets, visible labels that match names
Deaf and hard of hearing	Captions (on-screen speech text), transcripts (full text of audio/video)	Media alternatives (Chapter 4)
Cognitive and learning disabilities	Clear language, consistent layout, no time pressure	Predictable structure, clear errors, no memory tests
Everyone, sometimes ("situational")	Broken arm, bright sun, noisy train, slow network, old age	All of the above

The WHO estimates about 1.3 billion people — roughly one in six — live with a significant disability.

Why it matters

- **It is mostly not happening.** The WebAIM Million (top one million home pages, February 2026) found detectable WCAG failures on 95.9 % of them, ~56 errors per page. Six types made up 96 % of errors: low-contrast text (83.9 % of pages — the most common failure), missing alt text, missing form labels, empty links, empty buttons, missing page language. All are fixed by this chapter's basics; §3.10 builds a checker for exactly those six.

- **It is often the law.** The US ADA (Americans with Disabilities Act) and Section 508 (US government sites), the European Accessibility Act (applying since 28 June 2025) and similar laws require accessible services, usually by reference to WCAG level AA.
- **It is better engineering.** Semantic HTML gives keyboard support, focus handling and screen-reader output *for free*, and improves SEO, reader modes, voice assistants, testing (Testing Library finds elements by role — React book) and readability.

3.2 Landmarks

What it is

Landmarks are a page's big regions — banner, navigation, main, sidebar, footer. Screen-reader users list them and jump straight to one (NVDA's `D` key; VoiceOver's **rotor**, a menu of headings, links or landmarks). HTML's **sectioning elements** create landmarks automatically; the **role** is what a screen reader announces:

Element	Landmark role (ARIA)	Rules
`<header>`	`banner` — only when *not* inside `article`, `aside`, `main`, `nav` or `section`	Site header: logo, site nav, search. Inside an article it's just that article's header.
`<nav>`	`navigation`	Major navigation blocks only. With several, label each: `aria-label="Main"`, `"Breadcrumb"` (the Home › Shop › Boots trail showing where a page sits)
`<search>`	`search`	Wraps a search/filter form. Baseline 2023, widely available since April 2026 (before: `<form role="search">`).
`<main>`	`main`	The primary content, **one visible per page**; target of the skip link
`<aside>`	`complementary`	Related but separable: sidebar, related links, pull quote
`<footer>`	`contentinfo` — only when not inside `article`, `aside`, `main`, `nav` or `section`	Site footer: copyright, contact, legal links
`<section>`	`region` — **only if it has an accessible name** (the label a screen reader reads, e.g. via `aria-labelledby` pointing at its heading)	A thematic group with its own heading
`<form>`	`form` — only if it has an accessible name	
`<article>`	`article` (a document-structure role, not a landmark)	Self-contained: blog post, product card, comment, news story

article could be syndicated (republished in a feed) or reused alone; **section** is a chunk of something bigger with its own heading (a chapter, tab panel, "Reviews"); **div** has no meaning. Unsure between `section` and `div`? Ask "would this appear in a table of contents?" If not, `div`.

Example

`code/landmarks.html`, a small news page (the yellow tags in the screenshot are `aria-hidden="true"` labels for this book):

```html
<a class="skip" href="#main">Skip to main content</a>
<header>
  <a class="logo" href="/">Mountain Club</a>
  <nav aria-label="Main">
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/news" aria-current="page">News</a></li>
      <li><a href="/events">Events</a></li>
    </ul>
  </nav>
  <search>
    <form action="/search"><label for="q">Search</label> <input id="q" name="q" type="search"></form>
  </search>
</header>
<div class="wrap">
  <main id="main">
    <h1>Trail news</h1>
    <article>
      <header>
        <h2>Ridge trail reopens</h2>
        <p>By Sam Lee · <time datetime="2026-09-20">20 September 2026</time></p>
      </header>
      <p>After three months of repairs, the ridge trail is open again.</p>
      <section aria-labelledby="h-what">
        <h3 id="h-what">What changed</h3>
        <p>New steps on the steep part and two rest benches.</p>
      </section>
      <footer><p>Filed under <a href="/tags/trails">trails</a></p></footer>
    </article>
  </main>
  <aside aria-labelledby="h-side">
    <h2 id="h-side">Upcoming</h2>
    <ul><li>Night run, Fri</li><li>Hill repeats, Sun</li></ul>
  </aside>
</div>
<footer><small>© 2026 Mountain Club</small></footer>
```



Figure 3.1 — `code/landmarks.html`. The layout is CSS; the landmarks come from the elements.

Chrome's accessibility tree and heading outline (`code/01_landmarks_tree.mjs`, text and generic nodes skipped):

```
RootWebArea "Trail news — Mountain Club"
  link "Skip to main content"
  banner
    link "Mountain Club"
    navigation "Main"
      list
        link "Home"
        link "News"
        link "Events"
    search
      form
        searchbox "Search"
  main
    heading "Trail news"  level=1
    article
      sectionheader
        heading "Ridge trail reopens"  level=2
      region "What changed"
        heading "What changed"  level=3
      sectionfooter
        link "trails"
  complementary "Upcoming"
    heading "Upcoming"  level=2
    list
  contentinfo

Heading outline:
h1 Trail news
  h2 Ridge trail reopens
    h3 What changed
  h2 Upcoming
```

1. Top-level `<header>` / `<footer>` became `banner` / `contentinfo`; the article's own header and footer did *not* (`sectionheader` / `sectionfooter` aren't landmarks).
2. `<section>` became a `region` because `aria-labelledby` named it; `<aside>` got "Upcoming" the same way.
3. The `<nav>` is "Main navigation" thanks to `aria-label`; `aria-current="page"` marks the current link.
4. Everything except the skip link sits inside a landmark — aim for that, so nothing is orphaned outside the regions users jump between.

Common mistakes

- "Div soup": `<div class="header">`, `<div class="nav">` — zero landmarks.
- Several `<main>` elements, or none.
- `<section>` everywhere "because HTML5" — sections without headings add noise.
- Several unlabelled `<nav>`s: "navigation, navigation, navigation".

GOV.UK, the BBC and GitHub use this exact skeleton (skip link → `header` with `nav` → `main` → `footer`).

3.3 Headings and the outline

Headings are the other main navigation tool: users press `H` (NVDA/JAWS) or use the rotor to jump heading to heading, or list all headings as a table of contents (the outline above). The rules of §2.4 apply: one `h1`, no skipped levels going down, levels by structure, and `<section>` / `<article>` don't reset levels (the "outline algorithm" was never implemented and was removed).

Accessibility-specific points:

- Each landmark usually starts with a heading — hence naming a `section` or `aside` with `aria-labelledby` pointing at it.
- A region with no visible heading (the main nav) gets `aria-label`, not a fake heading.

3.4 The accessibility tree

What it is

Alongside the DOM, browsers build an **accessibility tree** — only what matters to AT — exposed through the OS accessibility API (UI Automation on Windows, NSAccessibility on macOS, AT-SPI on Linux). Screen readers never read your HTML; they read this tree.

Property	Question it answers	From
Role	What is it? (button, link, heading, textbox, checkbox, navigation…)	The element's implicit role (`<button>` → button) or an ARIA `role`
Name	What is it called? ("Save", "Search", "Close")	Content, `<label>`, `alt`, `aria-label`, `aria-labelledby` …
Description	Extra help text	`aria-describedby`, sometimes `title`
State / properties	What condition? (checked, expanded, disabled, invalid, required, level 2, current page)	Native attributes (`checked`, `disabled`, `required`) or ARIA states
Value	Current value of a field, slider, progress bar	The control's value

A screen reader speaks it as: *"Email, edit text, required, invalid entry, We only use it for your receipt."*

How the accessible name is computed

The W3C *Accessible Name and Description Computation* spec; simplified, the first that yields text wins:

1. `aria-labelledby` — text of the referenced element(s), in order (several ids allowed).
2. `aria-label` — a string on the element.
3. The native label: `<label>` for fields, `alt` for images, `<caption>` for tables, `<legend>` for fieldsets, `<figcaption>` for figures.
4. The element's content, for roles with "name from content" (buttons, links, headings, cells…).
5. `title` (and for inputs `placeholder`) as a last resort.

Hidden content (`display: none`, `hidden`, `aria-hidden="true"`) is left out — unless referenced directly by `aria-labelledby`.

Chrome's computation for `code/aria.html` (§3.6), from `code/03_names_and_states.mjs` (`StaticText` nodes omitted; the visually hidden "Rated 4 out of 5" text is one of them):

```
--- aria.html
RootWebArea "ARIA patterns"
  button "Close"
  button "Shopping cart 3 items"
  textbox "Password"  description="At least 12 characters."
  button "Shipping details"  expanded=true
  navigation "Breadcrumb"
    link "Home"
    link "Shop"
    link "Boots"
  button "Add to cart"
  status

after clicking "Shipping details" and "Add to cart": {
  expanded: 'false',
  panelHidden: true,
  status: 'Added. Cart has 4 items.'
}
```

- The "×" button is named **"Close"** (`aria-label` beat the content).
- The cart button combines two elements via `aria-labelledby`: **"Shopping cart 3 items"**.
- The password field's name is its `<label>`; its **description** comes from `aria-describedby`.

- The `aria-hidden` stars are gone; only the visually hidden text remains.

Inspecting it yourself

Chrome DevTools → Elements → **Accessibility** pane (or the full-page tree view); Firefox → **Accessibility** panel; Safari → Elements → Node → Accessibility.

In the interview

> **30-second answer:** "Browsers derive an accessibility tree from the DOM — each node has a role, an accessible name, maybe a description, states and a value — and expose it to screen readers through OS accessibility APIs. Native elements fill it in correctly; ARIA can override or add. The name comes, in order, from `aria-labelledby`, `aria-label`, native labels like `<label>` or `alt`, the content, and finally `title` or placeholder."

- *Does CSS affect the accessibility tree?* → Yes: `display: none` and `visibility: hidden` remove nodes; generated content (`::before`/`::after`, Chapter 6) can become part of names; `opacity: 0` hides nothing.
- *Is a `<div>` with a click handler a button to a screen reader?* → No: no role, no button name, not focusable (§3.7).

3.5 Alt text

What it is

`alt` is an image's **text alternative**: what a screen reader speaks, what search engines index, what shows if the image fails. It is WCAG success criterion 1.1.1 (Non-text Content, level A) and a top-six failure.

How to decide — ask "what is this image *for*?"

Ask in order	If yes →
1. Purely decorative, or nearby text already says it?	`alt=""` (empty — image ignored)
2. Content of a link or button?	Describe the **destination or action** ("Mountain Club home", "Search")
3. Complex (chart, diagram, map)?	Short `alt` naming it + the full information in nearby text or `<figcaption>`
4. An image of text?	`alt` repeats the text — better, use real text styled with CSS
5. Otherwise (a meaningful photo/illustration)	What matters about it **in this context**, briefly

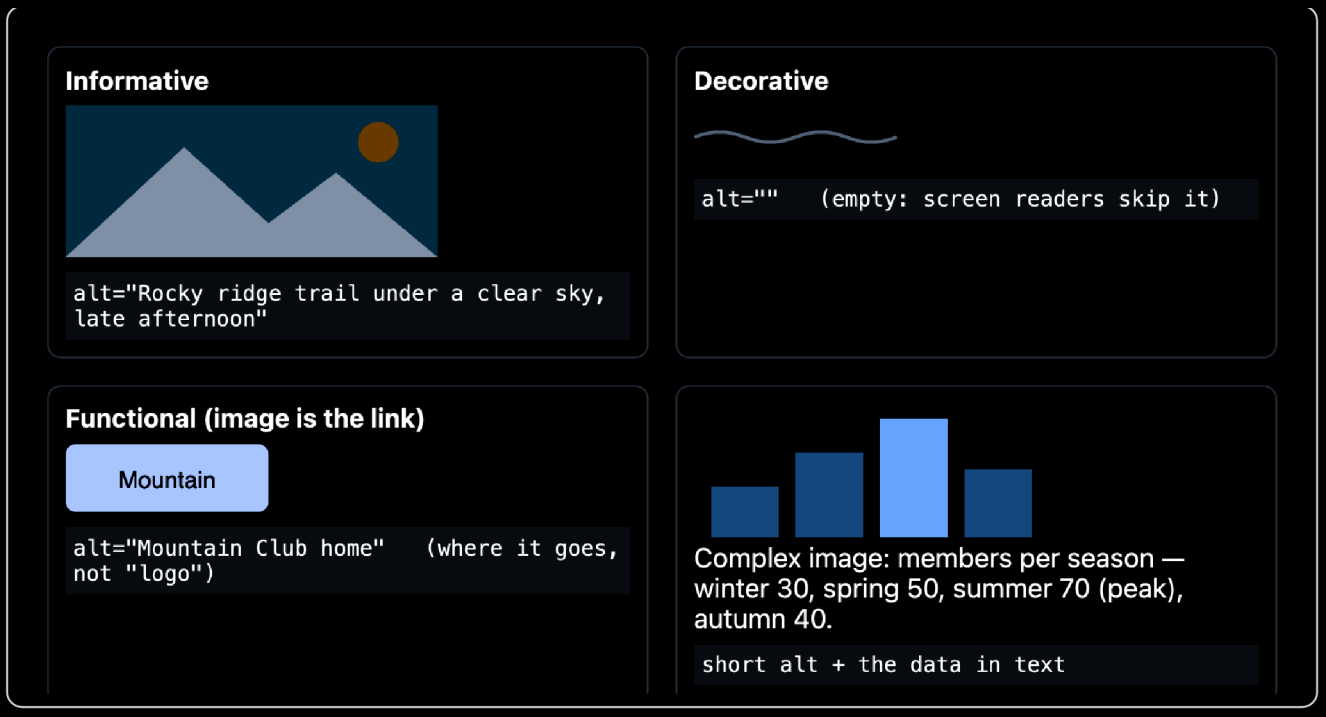


Figure 3.2 — `code/alt-text.html`: the four most common cases.

```
<img src="ridge.jpg" alt="Rocky ridge trail under a clear sky, late afternoon">      <!-- informative -->
<img src="squiggle.svg" alt="">                                                       <!-- decorative -->
<a href="/"><img src="logo.svg" alt="Mountain Club home"></a>                         <!-- functional -->
<figure>                                                                              <!-- complex -->
  <img src="members.svg" alt="Bar chart of club members per season">
  <figcaption>Complex image: members per season — winter 30, spring 50, summer 70 (peak), autumn 40.</figcaption>
</figure>
```

Chrome's view (from `03_names_and_states.mjs`):

```
--- alt-text.html (images only)
image "Rocky ridge trail under a clear sky, late afternoon"
image "Mountain Club home"  (inside link "Mountain Club home")
image "Bar chart of club members per season"
decorative <img alt=""> in tree? false
```

The empty-alt image is gone from the tree; the linked image names the link.

Rules of thumb

- **Missing `alt` ≠ empty `alt`**: missing → screen readers may read the file name ("IMG underscore 2041 dot jpg"); empty → skipped.
- Don't start with "Image of…" — the role already says "image" (do say "Photo", "Screenshot" when the *type* matters).
- Short; what matters **in context** — the same photo is "Dr. Ana Ruiz" on a staff page, "Ana Ruiz holding the award trophy" in a news story.
- Don't repeat the caption or adjacent text; if the text already says it, `alt=""`.
- Icons beside visible text ("🗑 Delete") are decorative; icon-only buttons need a name on the button (`aria-label`).
- CSS background images have no alt: never put meaningful content in them.
- Inline `<svg>`: `role="img"` + `aria-label` (or a `<title>` child) when informative, `aria-hidden="true"` when decorative (Chapter 4).

3.6 ARIA

What it is

WAI-ARIA (Web Accessibility Initiative — Accessible Rich Internet Applications; "ARIA") is a set of attributes that change or add to what the accessibility tree says:

- **Roles** — `role="button"`, `"tablist"`, `"dialog"`, `"alert"` …
- **Properties** (mostly stable facts) — `aria-label`, `aria-labelledby`, `aria-describedby`, `aria-controls`, `aria-required`, `aria-haspopup` …
- **States** (change with interaction) — `aria-expanded`, `aria-checked`, `aria-pressed`, `aria-selected`, `aria-invalid`, `aria-disabled`, `aria-current`, `aria-hidden` …

The crucial fact: **ARIA only changes what is announced. It adds no behaviour** — no focusability, keyboard handling or styling. `role="button"` on a `<div>` makes a screen reader *say* "button"; you still must make it focusable and handle Enter and Space.

The first rule of ARIA (and the other four)

From the W3C "Using ARIA" note:

1. **Use native HTML first.** If an element/attribute with the needed meaning and behaviour exists (`<button>`, `<a href>`, `<input type="checkbox">`, `<details>`, `<dialog>`, `required`, `disabled`), use it.
2. **Don't change native semantics** unless you must (`<h2 role="tab">` is wrong; put a `<button role="tab">` inside the `<h2>`).
3. **Every interactive ARIA control (*widget* — a custom component such as tabs or a menu) must work with the keyboard.**
4. **Never put `role="presentation"` / `"none"` or `aria-hidden="true"` on a focusable element** — users would tab to something that "doesn't exist".
5. **Every interactive element must have an accessible name.**

No ARIA is better than bad ARIA: WebAIM's surveys consistently find home pages *with* ARIA have more detected errors on average than pages without — usually ARIA patching div-based widgets instead of native elements.

Useful ARIA — the patterns in `aria.html`

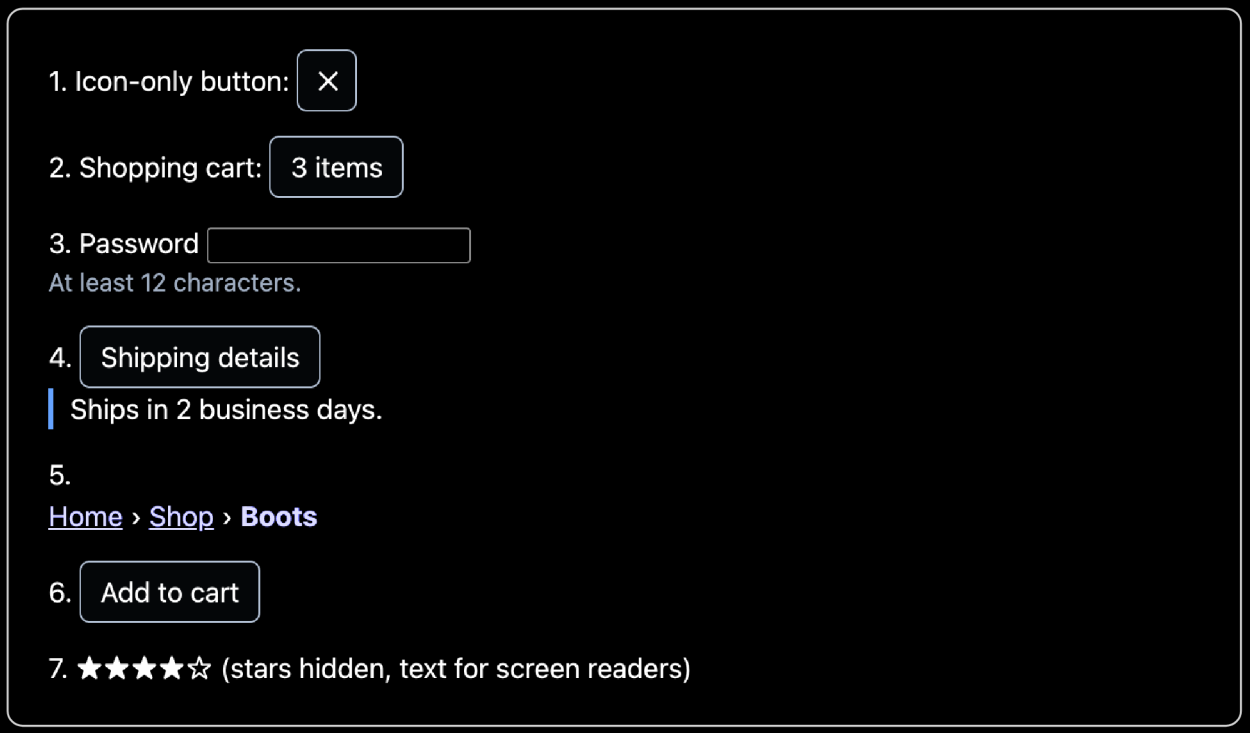


Figure 3.3 — `code/aria.html`. Visually ordinary — the differences are in the accessibility tree (§3.4).

```html
<!-- 1. Name an icon-only control -->
<button type="button" aria-label="Close">x</button>

<!-- 2. Build a name from visible text elsewhere (ids, in order) -->
<span id="cart-h">Shopping cart</span>:
<button type="button" aria-labelledby="cart-h cart-n"><span id="cart-n">3 items</span></button>

<!-- 3. Add help text as a description -->
<label for="pw">Password</label> <input id="pw" type="password" aria-describedby="pw-hint">
<div id="pw-hint">At least 12 characters.</div>

<!-- 4. Disclosure: state that JavaScript keeps in sync -->
<button type="button" aria-expanded="true" aria-controls="panel" id="toggle">Shipping details</button>
<div id="panel">Ships in 2 business days.</div>

<!-- 5. Mark the current item in a set of links -->
<nav aria-label="Breadcrumb"> … <a href="/shop/boots" aria-current="page">Boots</a></nav>

<!-- 6. Live region: announce changes without moving focus -->
<button type="button" id="add">Add to cart</button> <span role="status" id="live"></span>

<!-- 7. Hide decoration, provide a text equivalent -->
<span aria-hidden="true">★★★★☆</span><span class="visually-hidden">Rated 4 out of 5</span>
```

```js
// the only JavaScript: keep aria-expanded true to the UI, and write into the live region
t.addEventListener('click', () => { const open = t.getAttribute('aria-expanded') === 'true';
  t.setAttribute('aria-expanded', String(!open)); p.hidden = open; });
document.getElementById('add').addEventListener('click', () => { live.textContent = 'Added. Cart has 4 items.';
});
```

Attribute	Use when	Watch out
`aria-label`	No visible text (icon buttons, several `<nav>`s)	Some translation tools skip it; voice-control users can't see it — prefer visible text
`aria-labelledby`	Visible text elsewhere names it	Takes ids; overrides everything else (wins over `aria-label`)
`aria-describedby`	Hints, format rules, error messages	Read after the name, sometimes after a pause
`aria-expanded`	Buttons that show/hide something (menus, accordions, disclosures)	Your script must update it
`aria-current`	Current page/step/date in a set	`page`, `step`, `location`, `date`, `time`, `true`
`aria-hidden="true"`	Decorative or duplicated content	Removes the whole subtree from the tree; doesn't hide visually or stop focus — never on focusable things
`aria-live` / `role="status"` / `role="alert"`	Content changing without a page load (toasts, counters, errors)	Region must **exist before** the text changes; `status` = polite (waits until idle), `alert` = assertive (interrupts) — urgent errors only
`aria-invalid`, `aria-required`	Errors / required fields when native attributes don't fit	Native `required` already implies `aria-required`

> **Key term — live region:** an element whose text changes are read out automatically while focus stays put (`aria-live`, `role="status"`, `role="alert"`, changed later by JS). It's how a blind user hears "Item added" or "Saved" after an AJAX save (a background request without reload) without being moved around the page.

The **visually-hidden** pattern (`.visually-hidden` / `.sr-only`) hides text on screen but keeps it in the accessibility tree:

```
.visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden;
                   clip-path: inset(50%); white-space: nowrap; }
```

When not to use it

- To re-create a native control (`<div role="checkbox">`).
- To "fix" headings (`<div role="heading" aria-level="2">`) when you can edit the HTML.
- `aria-label` on a role-less `<div>` / `<span>` : many screen readers ignore names on generic elements.
- `role="application"` — switches off the screen reader's normal reading keys; almost never appropriate.

In the interview

> **30-second answer:** "ARIA sets roles, states and properties in the accessibility tree when HTML can't express something — `aria-expanded` on a disclosure button, `aria-live` regions, naming an icon button. But the first rule is to use native HTML when it exists, because ARIA only changes semantics: no focus, keyboard or behaviour. A `<button>` gives me all of that free."

- *`aria-label` vs `aria-labelledby`?* → A string on the element vs a pointer to other elements' visible text (wins if both exist).
- *`role="status"` vs `role="alert"`?* → Both live regions; status is polite, alert assertive — urgent errors only.
- *What does `aria-hidden="true"` do?* → Removes the element and descendants from the accessibility tree only.

3.7 Keyboard and focus

What it is

Keyboard-only users, screen-reader users (screen readers are keyboard-driven), and switch and voice users (whose tools emulate keys) never use a mouse. **Focus** is the element receiving keyboard input: **Tab** forward, **Shift+Tab** back, **Enter** activates links and buttons, **Space** activates buttons and toggles checkboxes, **arrows** move within composite widgets (radio groups, menus, tabs, selects), **Escape** closes dialogs and popups.

WCAG requires everything to work by keyboard (2.1.1), no keyboard traps (2.1.2 — places Tab can't leave), logical order (2.4.3), and a visible focus indicator (2.4.7) not hidden behind sticky headers (2.4.11, new in 2.2).

The div-button problem, measured

`code/div-vs-button.html` has three identical-looking controls:

```
<div class="btn" onclick="save()">Save (div)</div>
<button class="btn" type="button" onclick="save()">Save (button)</button>
<div class="btn" role="button" tabindex="0" onclick="save()">Save (div + role + tabindex)</div>
<p id="out" role="status"></p>
```



Figure 3.4 — Pixel-identical. The screenshot can't tell you which one works.

`code/02_div_vs_button.mjs` prints Chrome's tree, presses Tab four times, then presses Enter/Space on the `<button>` and Enter on the role=button div:

```
Accessibility tree:
RootWebArea "div vs button"
  StaticText "All three look identical. Only the middle one is a real button."
  StaticText "Save (div)"
  button "Save (button)"
  button "Save (div + role + tabindex)"
  status

Pressing Tab 4 times:
  Tab 1 -> Save (button)
  Tab 2 -> Save (div + role + tabindex)
  Tab 3 -> (body — focus left the page)
  Tab 4 -> Save (button)

button     + Enter: status = "Saved!"

button     + Space: status = "Saved!"

div + role + Enter: status = ""
```

1. The plain `<div>` is **just text** and **never reachable** by Tab.
2. The `<button>` is announced, focusable, and works with **Enter and Space** — free.
3. The `div` with `role` + `tabindex` is announced and focusable, but **Enter does nothing**: `onclick` on a div fires only for pointer clicks (mouse, touch, pen). You'd need a `keydown` handler for Enter *and* Space, disabled-state handling and form-submission behaviour — or just `<button>`.

`tabindex`

Value	Effect	Use
(none)	Natively interactive elements (`a[href]`, `button`, `input`, `select`, `textarea`, `summary`…) are focusable in DOM order	Default — keep DOM order = visual order
`tabindex="0"`	Adds a non-interactive element to the Tab order at its DOM position	Custom widgets (with role and key handlers), scrollable regions
`tabindex="-1"`	Focusable by script (`el.focus()`) or as a link target, not by Tab	Skip-link targets, headings you move focus to, items in roving-focus widgets (one item tabbable, arrows move among the rest), dialog containers
`tabindex="1"` +	Jumps ahead of everything in the Tab order	**Never** — breaks natural order, unmaintainable

Keep **DOM order** (source order) **= visual order**: CSS `order`, `flex-direction: row-reverse`, grid placement and absolute positioning move boxes but not the Tab sequence (Chapters 8–9).

Visible focus

Browsers draw a focus ring by default; the commonest way to break keyboard access is `outline: none` in a reset stylesheet. Style it with `:focus-visible` instead — it matches when the browser's heuristics say an indicator is useful (keyboard navigation), not mouse clicks on buttons; `:focus` matches any focus:

```
:focus-visible { outline: 3px solid #c2410c; outline-offset: 2px; }
.bad:focus { outline: none; }     /* never do this without a replacement */
```

The ring needs ≥ 3:1 contrast against the background (non-text contrast, 1.4.11; §3.9) — this dark orange is ~5:1 on white; a light amber would be ~2:1.

Skip links

A **skip link** — the first focusable element — jumps past the header to `<main>`, so keyboard users don't Tab through it on every page (WCAG 2.4.1 Bypass Blocks; landmarks and headings help too):

```html
<a class="skip" href="#main">Skip to main content</a>
...
<main id="main" tabindex="-1">…</main>
```

```css
.skip { position: absolute; left: 1rem; top: -3rem; }   /* off-screen until focused */
.skip:focus { top: 0; }
```

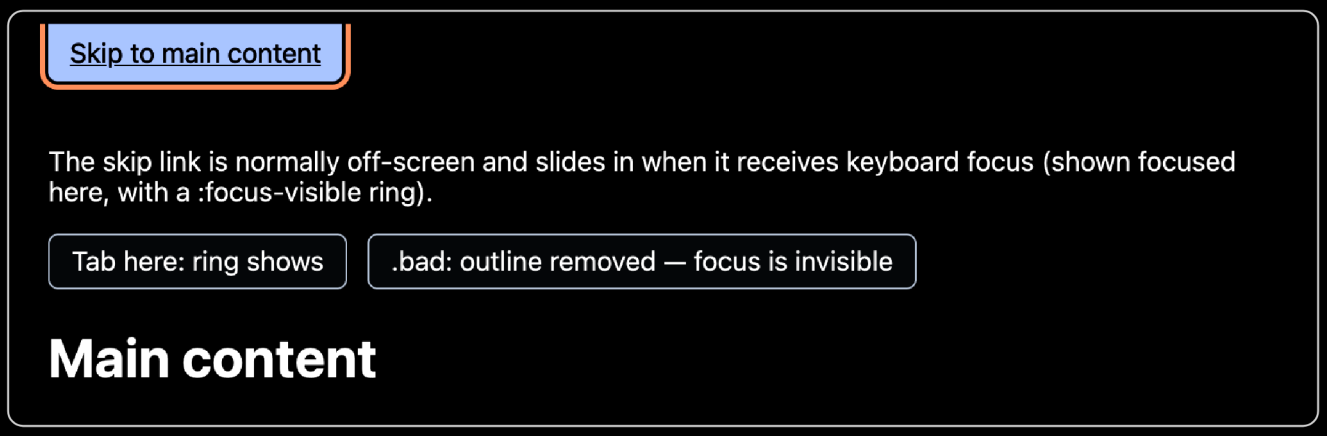


Figure 3.5 — `code/focus.html` with the skip link focused (focused on load for the screenshot).

Managing focus in dynamic UIs

- **Dialogs**: move focus in; keep Tab inside while open; Escape closes; return focus to the opener. Native `<dialog>` with `showModal()` makes the rest inert and handles Escape.
- `inert`: makes a subtree unfocusable and hidden from AT — content behind a custom modal or an off-canvas (slide-in) menu.
- **Single-page app navigation** (JS swaps content instead of loading a page): after a route change, move focus to the new `<h1>` (`tabindex="-1"`) or announce it; otherwise screen-reader users don't know anything happened (React book).
- **Deleting the focused item**: move focus to a sensible neighbour, not the page top.
- **Target size**: WCAG 2.2 2.5.8 (AA) — pointer targets ≥ **24×24 CSS px** (or enough spacing).

Common mistakes

- `outline: none` with no replacement; positive `tabindex`.
- Clickable `div`s/`span`s; links used as buttons and vice versa.
- Custom dropdowns that can't open by keyboard or trap focus; hover-only menus that never open on focus.

In the interview

30-second answer: "Everything interactive must be reachable and operable by keyboard, in a logical order, with visible focus. Native elements give me that; I use `tabindex=0` only for genuine custom widgets and `-1` for programmatic focus targets, never positive values. I style focus with `:focus-visible` instead of removing outlines, add a skip link, and manage focus for dialogs and route changes."

- *Why not `tabindex="5"`?* → It jumps ahead of all `tabindex=0`/native elements, so order no longer matches the page.
- *How do you test keyboard access?* → Unplug the mouse and run the 5-minute test (§3.10).

3.8 Accessible forms

Forms are where failures cost most — a user who can't identify a field can't sign up, pay or apply. Full coverage in Chapter 5; the accessibility essentials:

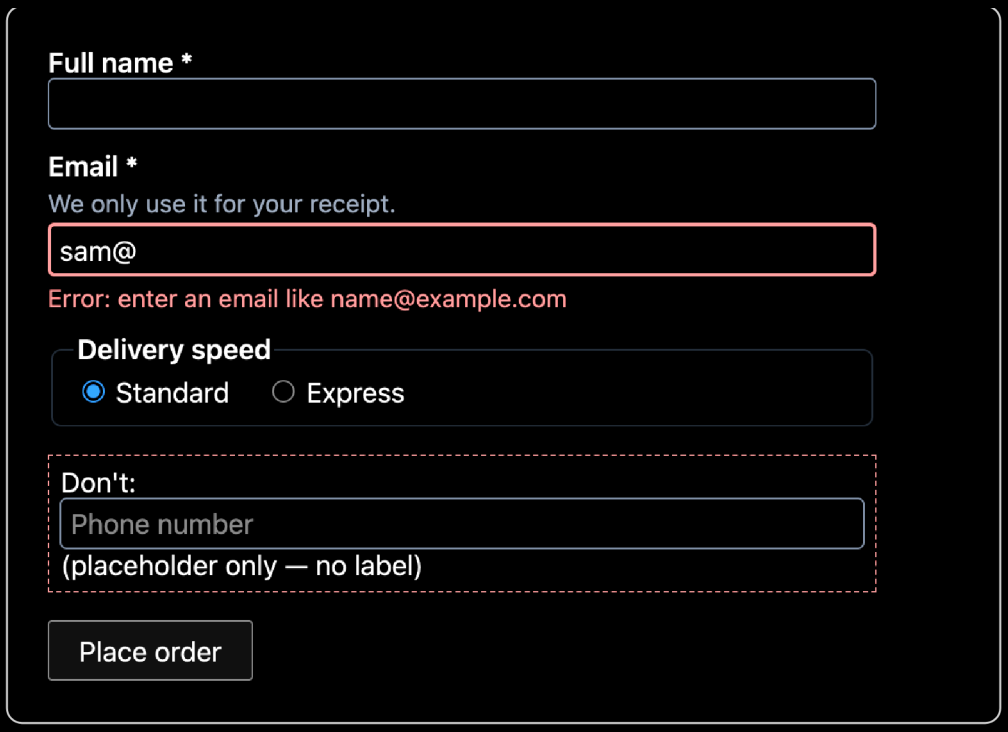


Figure 3.6 — `code/form-a11y.html`.

```
<label for="name">Full name <span aria-hidden="true">*</span></label>
<input id="name" type="text" name="name" autocomplete="name" required>

<label for="email">Email <span aria-hidden="true">*</span></label>
<p class="hint" id="email-hint">We only use it for your receipt.</p>
<input id="email" type="email" name="email" autocomplete="email" required
       aria-invalid="true" aria-describedby="email-hint email-err" value="sam@">
<p class="error" id="email-err">Error: enter an email like name@example.com</p>

<fieldset>
  <legend>Delivery speed</legend>
  <input type="radio" id="std" name="speed" checked><label for="std">Standard</label>
  <input type="radio" id="exp" name="speed"><label for="exp">Express</label>
</fieldset>

Don't: <input type="text" placeholder="Phone number">   <!-- placeholder only, no label -->
```

What Chrome exposes (`code/04_form_tree.mjs`):

```
RootWebArea "Accessible form"
  form
    textbox "Full name"  required=true
    textbox "Email"  required=true invalid="true" value="sam@" description="We only use it for your receipt.
Error: enter an email like name@example.com"
    group "Delivery speed"
      Legend
      radio "Standard"  checked=true
      radio "Express"  checked=false
    textbox "Phone number"
    button "Place order"

After typing into the placeholder-only field, the visible hint is gone: { value: '555', placeholderShown: false }
```

1. **Every field has a `<label>`**, tied by `for` / `id` or by wrapping the input. It becomes the name *and* enlarges the click target (clicking "Standard" checks the radio).
2. **Group related controls** (radios, checkboxes, address parts) in `<fieldset>` + `<legend>`; the legend names the group ("Delivery speed, grouping").
3. **Hints and errors** link via `aria-describedby`; `aria-invalid="true"` marks the error. Error text says what's wrong and how to fix it, not by colour alone (the word "Error:" and the thicker border — WCAG 1.4.1 Use of Color, 3.3.1 Error Identification, 3.3.3 Error Suggestion).

4. **Required**: native `required` (exposed as `required=true`); the visual `*` is `aria-hidden` because the state is already announced — explain the asterisk once at the top.
5. **Placeholders are not labels.** Chrome falls back to the placeholder as a name, but it vanishes on typing (see output), is usually low-contrast, and some AT ignores it.
6. `autocomplete` tokens (`name`, `email`, `street-address`, `one-time-code`…) let browsers and password managers fill fields — WCAG 1.3.5 (Identify Input Purpose), a big help for motor and memory impairments. WCAG 2.2 adds 3.3.7 Redundant Entry (don't make people retype information) and 3.3.8 Accessible Authentication (allow password managers and paste; no memory/puzzle tests without an alternative).
7. On submit with errors, move focus to an error summary or the first invalid field.

3.9 WCAG 2.2

What it is

The **Web Content Accessibility Guidelines** (WCAG, W3C) are the international standard laws and contracts point to. **WCAG 2.2** became a W3C Recommendation (final standard status) on 5 October 2023. It is backwards compatible: meeting 2.2 also meets 2.1 and 2.0.

- **4 principles — POUR**: **Perceivable**, **Operable**, **Understandable**, **Robust**.
- **13 guidelines** (e.g. 1.1 Text Alternatives, 2.1 Keyboard Accessible, 3.3 Input Assistance).
- **Success criteria** (SC) — testable statements numbered like `1.4.3`, each at a **level**:
  - **A** — minimum; without it some users are completely blocked.
  - **AA** — the usual target for laws and policies; "conform to WCAG 2.2 AA" = every A and AA criterion.
  - **AAA** — enhanced; not recommended as a blanket site requirement because some criteria can't be met for all content.

Criteria you will meet most often

Principle	SC	Level	In plain words
Perceivable	1.1.1 Non-text Content	A	Images, icons and controls have text alternatives (§3.5)
	1.3.1 Info and Relationships	A	Visual structure is in the markup: headings, lists, labels, tables
	1.3.5 Identify Input Purpose	AA	Personal-data fields use `autocomplete`
	1.4.1 Use of Color	A	Colour is not the only signal
	1.4.3 Contrast (Minimum)	AA	Text ≥ 4.5:1 (large text ≥ 3:1)
	1.4.4 Resize Text	AA	Works at 200 % text size
	1.4.10 Reflow	AA	Usable at 320 CSS px wide (400 % zoom) without horizontal scrolling
	1.4.11 Non-text Contrast	AA	UI parts and meaningful graphics (input borders, focus rings, icons) ≥ 3:1
Operable	2.1.1 Keyboard	A	Everything works from a keyboard
	2.1.2 No Keyboard Trap	A	Focus can always move away
	2.4.1 Bypass Blocks	A	Skip links / landmarks
	2.4.2 Page Titled	A	Descriptive `<title>`
	2.4.3 Focus Order	A	Logical Tab order
	2.4.4 Link Purpose (In Context)	A	Link text makes sense
	2.4.6 Headings and Labels	AA	Headings and labels describe topic or purpose
	2.4.7 Focus Visible	AA	A visible focus indicator
Understandable	3.1.1 Language of Page	A	`<html lang>`
	3.3.1 Error Identification	A	Errors described in text
	3.3.2 Labels or Instructions	A	Inputs have labels/instructions
Robust	4.1.2 Name, Role, Value	A	Every UI component exposes correct name, role, state (§3.4)
	4.1.3 Status Messages	AA	Status messages announced without moving focus (live regions)

What's new in WCAG 2.2

Nine new success criteria:

SC	Level	Requirement
2.4.11 Focus Not Obscured (Minimum)	AA	A focused element isn't entirely hidden by sticky headers, cookie banners, etc.
2.4.12 Focus Not Obscured (Enhanced)	AAA	…not hidden at all
2.4.13 Focus Appearance	AAA	Focus indicator big enough with enough contrast
2.5.7 Dragging Movements	AA	Anything done by dragging also works with single clicks/taps
2.5.8 Target Size (Minimum)	AA	Pointer targets ≥ 24×24 CSS px (or enough spacing)
3.2.6 Consistent Help	A	Help links/contact info in the same place across pages
3.3.7 Redundant Entry	A	Don't ask for the same information twice in a process
3.3.8 Accessible Authentication (Minimum)	AA	No cognitive tests (remembering, transcribing, puzzles) to log in unless an alternative or aid exists — allow paste and password managers
3.3.9 Accessible Authentication (Enhanced)	AAA	Stricter (no object/image recognition exceptions)

One removal: **4.1.1 Parsing** is obsolete — browsers and AT now handle markup errors consistently (Chapter 1's parsing algorithm). Validation is still good practice (§2.10).

Colour contrast — the formula

Contrast ratio compares the **relative luminance** (perceived brightness, 0 = black to 1 = white) of text and background:

1. Convert each sRGB channel (the screen colour space's red/green/blue parts) from 0–255 to 0–1 and undo the gamma curve (screens store brightness on a curve; this recovers linear light): `c ≤ 0.04045 ? c/12.92 : ((c+0.055)/1.055)^2.4`.
2. `L = 0.2126 R + 0.7152 G + 0.0722 B` — green weighs most because eyes are most sensitive to it.
3. Ratio = `(L_lighter + 0.05) / (L_darker + 0.05)`, from 1:1 to 21:1 (black on white).

Level	Normal text	Large text (≥ 24 px, or ≥ 18.66 px bold)	UI components & graphics
AA	4.5:1	3:1	3:1 (1.4.11)
AAA	7:1	4.5:1	—

```mjs
// code/contrast_lib.mjs
export function luminance(hex) {
  const [r, g, b] = hex.replace('#', '').match(/../g).map(h => parseInt(h, 16) / 255)
    .map(c => (c <= 0.04045 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4));   // undo sRGB gamma
  return 0.2126 * r + 0.7152 * g + 0.0722 * b;                          // green looks brightest to the eye
}
export function contrast(fg, bg) {
  const [a, b] = [luminance(fg), luminance(bg)].sort((x, y) => y - x);
  return (a + 0.05) / (b + 0.05);
}
```

`code/05_contrast.mjs` output:

```
pair                    ratio   normal text   large text / UI
black on white          21.00:1 pass AA       pass AA | AAA ok
mid grey on white       4.54:1  pass AA       pass AA
one step lighter        4.48:1  FAIL          pass AA
light grey placeholder  2.85:1  FAIL          FAIL
white on blue-500       3.68:1  FAIL          pass AA
white on blue-700       6.70:1  pass AA       pass AA
error red on white      6.47:1  pass AA       pass AA
white on amber          2.15:1  FAIL          FAIL
```

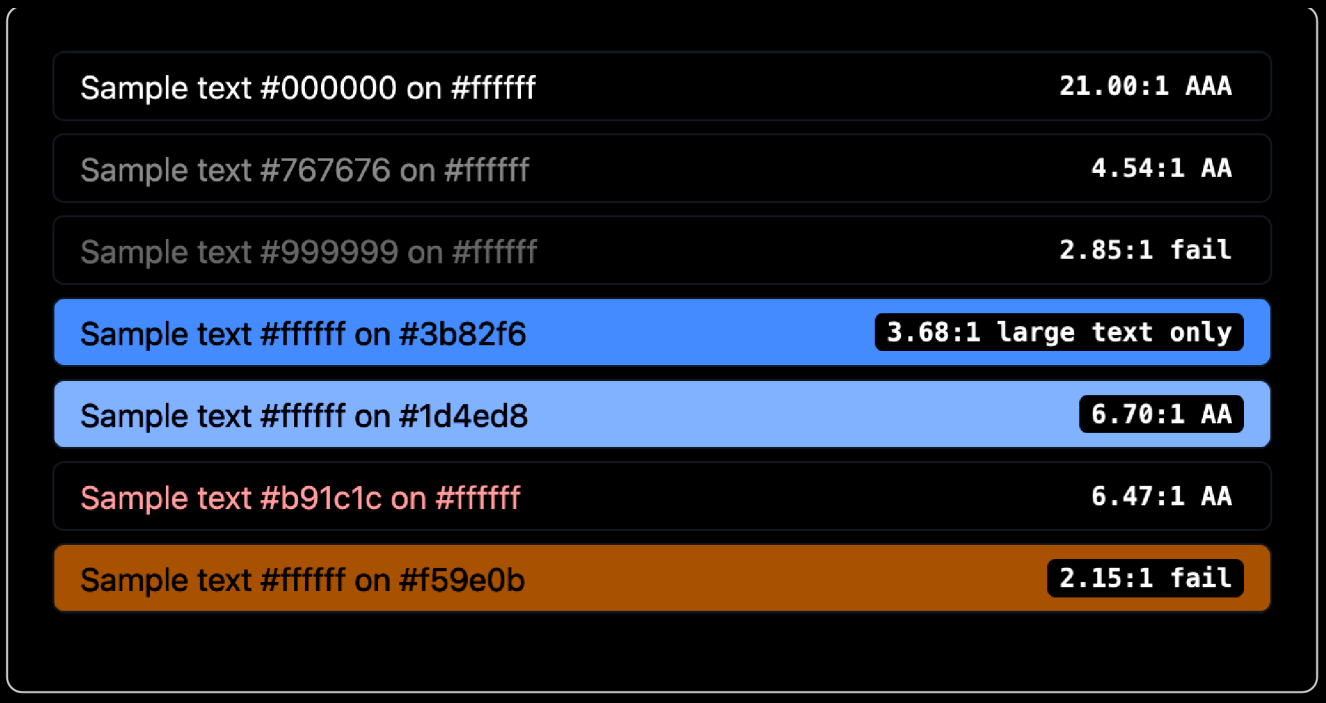


Figure 3.7 — `code/contrast.html` computes each ratio in the browser with the same formula.

Lessons: `#767676` is the lightest grey passing AA on white (4.54:1); one step lighter fails. White on mid-blue `#3b82f6` (a very common button colour) fails for normal text — this book's Chapter 1 page used it and switched to `#1d4ed8`. White on amber/yellow almost always fails. Thresholds are strict: 4.49:1 fails. DevTools' colour picker shows the ratio and AA/AAA lines live.

Self-check

1. Minimum contrast for 16 px body text at AA? → 4.5:1.
2. Name two criteria new in 2.2, and the one removed. → Target Size (Minimum) 2.5.8, Focus Not Obscured (Minimum) 2.4.11 (also Dragging Movements, Consistent Help, Redundant Entry, Accessible Authentication); 4.1.1 Parsing removed.

3.10 Testing tools

How it works — three layers

1. **Automated checkers** quickly find mechanical problems: missing alt/labels/`lang`, contrast, empty links/buttons, invalid ARIA. They can't judge whether alt text is *good*, focus order sensible, or a custom widget usable — estimates range from roughly a third of WCAG criteria to just over half of real-world issues by volume.
2. **Manual keyboard testing** — no tools needed.
3. **Screen-reader testing** — hear what users hear.

Tool	Type	Notes
axe DevTools / **axe-core** (Deque)	Extension; JS library for CI with Playwright, Cypress, Jest (`jest-axe`)	The engine behind many tools; designed for no false positives
Lighthouse	Built into Chrome DevTools	Accessibility score from a subset of axe rules; also performance/SEO (Chapter 14)
WAVE (WebAIM)	Extension / website	Icons drawn on the page — good for learning
DevTools Accessibility pane	Chrome/Firefox/Safari	Role, name, states; Firefox simulates colour-vision deficiencies and shows Tab order
W3C Nu checker / html-validate	Validators	Structural errors (§2.10)
eslint-plugin-jsx-a11y (lint rules), **Testing Library** `getByRole`	Dev-time	Catch issues in React code; role-based test queries double as accessibility tests

Screen reader	Platform	Cost	Start / key tips
NVDA	Windows	Free (open source)	`Insert` or `CapsLock` = NVDA key; `H` next heading, `D` next landmark, `Tab` next focusable
JAWS	Windows	Commercial	Similar single-key navigation; common in workplaces
VoiceOver	macOS, iOS	Built in	macOS: Cmd+F5 toggles; VO = Ctrl+Option; VO+U opens the rotor (headings, landmarks, links)
TalkBack	Android	Built in	Swipe right/left to move; double-tap to activate
Narrator	Windows	Built in	Basic testing

Test the common pairings: NVDA or JAWS with Chrome/Firefox/Edge, VoiceOver with Safari.

A 5-minute keyboard test

1. Click the address bar, then Tab. Is the first stop a working skip link?
2. Can you **see** focus at every step? Is it ever hidden behind a sticky header?
3. Does the order follow the visual layout?
4. Can you operate every control (Enter/Space/arrows), open and close every menu and dialog (Escape), and never get stuck?
5. Zoom to 200 % and 400 %: any overlap or horizontal scrolling?

A mini automated audit

`code/06_mini_audit.mjs` implements a handful of axe-style rules — covering all six WebAIM Million top failures — with jsdom and `dom-accessibility-api` (a JS implementation of the accessible-name spec, used by Testing Library): for each interactive element it calls `computeAccessibleName()` and `getRole()` and flags missing names, placeholder-only fields, positive `tabindex`, unfocusable `role=button`, and focusables inside `aria-hidden`; plus `html lang`, empty headings, `img` alt, broken `aria-describedby`/`labelledby` ids, orphan labels and inline-colour contrast. Run against a deliberately bad sign-up form:

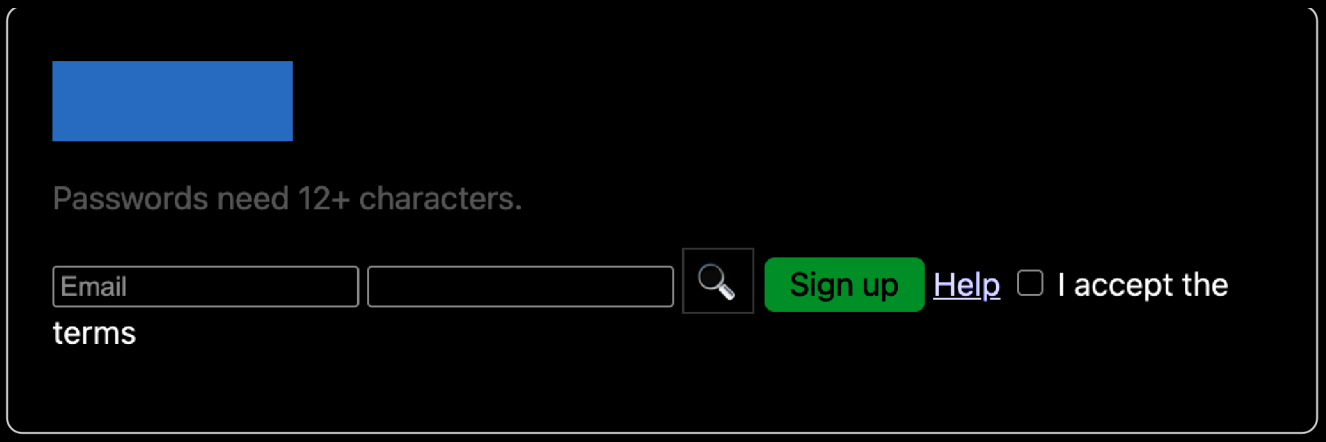


Figure 3.8 — `code/bad-a11y.html`. Looks usable — with a mouse and good eyesight.

```
bad-a11y.html: 12 issue(s)
rule                WCAG    element / problem
html-has-lang       3.1.1   <html> no lang attribute
empty-heading       BP      <h1> heading has no text
image-alt           1.1.1   <img> missing alt (use alt="" if decorative)
label               3.3.2   <input> only a placeholder ("Email"), no label
accessible-name     4.1.2   <input> password field has no accessible name
tabindex            2.4.3   <button> tabindex="3" breaks the natural focus order
focusable-role      2.1.1   <div> role="button" but not keyboard-focusable
aria-hidden-focus   4.1.2   <a> focusable element inside aria-hidden
accessible-name     4.1.2   <input> checkbox has no accessible name
aria-valid-ref      4.1.2   <input> references missing id "tos-text"
orphan-label        1.3.1   <label> "I accept the terms" is not tied to any control
color-contrast      1.4.3   <p> #aaaaaa on #ffffff is 2.32:1 (needs 4.5:1)

form-a11y.html: 1 issue(s)
rule                WCAG    element / problem
label               3.3.2   <input> only a placeholder ("Phone number"), no label
```

(`BP` = best practice, not a specific criterion.) The good form's only finding is its deliberate "Don't" example. The tool **cannot** know whether "🔍" is a good button name (a screen reader may say "magnifying glass tilted left"), whether alt texts are meaningful, or whether Tab order matches the layout — that's what the keyboard and screen-reader passes are for. In real projects use axe-core, not hand-written rules.

In the interview

30-second answer: "I layer it: axe or Lighthouse in development and CI catch missing labels, alt text and contrast; then a manual keyboard pass for focus order, visibility and traps; then a screen-reader pass — VoiceOver with Safari, NVDA with Chrome or Firefox — on key flows. Automation only finds part of the issues, so the manual steps aren't optional."

- *Can Lighthouse give 100 and the page still be inaccessible?* → Yes; it checks a subset of rules and nothing about meaning, order or usability.
- *How do you keep regressions out?* → axe-core in component/end-to-end tests, lint rules, role-based test queries, and accessibility in the definition of done.

Chapter 3 TL;DR

- **Semantic HTML is the foundation of accessibility**: native elements bring roles, names, states, focus and keyboard behaviour that assistive technology understands.
- **Landmarks**: top-level `header` → banner, `nav` → navigation (label multiples), `search`, one `main`, `aside` → complementary, top-level `footer` → contentinfo; `section` is a region only when named; `article` = self-contained.

- **Headings** form the navigable outline; no outline algorithm exists.
- **Accessibility tree** = role + name + description + states + value; names come from `aria-labelledby` → `aria-label` → native label/`alt` → content → `title`/placeholder.
- **Alt text**: purpose in context; `alt=""` for decoration; destination for linked images; complex images get a text equivalent.
- **ARIA**: first rule — use native HTML. ARIA changes semantics only, never behaviour. Use it for names, descriptions, states (`aria-expanded`, `aria-current`, `aria-invalid`), live regions (`role="status"`), and hiding decoration (`aria-hidden`).
- **Keyboard**: everything reachable and operable; DOM order = visual order; `tabindex` 0 / -1 only; `:focus-visible` rings with ≥ 3:1 contrast; skip link; manage focus for dialogs and route changes; targets ≥ 24×24 px.
- **Forms**: real `<label>`s, `fieldset`/`legend`, `aria-describedby` for hints and errors, `aria-invalid`, native `required`, `autocomplete`; placeholders are not labels.
- **WCAG 2.2 AA** is the usual target: POUR, 13 guidelines, A/AA/AAA; contrast 4.5:1 text, 3:1 large text and UI; nine new criteria, 4.1.1 removed.
- **Test in layers**: axe/Lighthouse/WAVE → keyboard pass → screen reader (NVDA, JAWS, VoiceOver, TalkBack).

Next: Chapter 4 — Images, Media and Embeds.